

Master Complexity Table -- Deep Analysis Report

Corpus: 6,117 unique real-user questions (4 elicitation stages, 8 personas, 96 participants)

Classifier: C

How to Read This Report

This report analyses a corpus of 6,117 natural language questions collected from real building users via Amazon MTurk. Participants were asked to imagine they could talk to their smart building and write down questions they would want answered. Questions were collected across four structured stages -- open-ended, sensor-prompted, scenario-based, and goal-oriented -- from 96 participants representing 8 personas (students, occupants, guests, health-and-safety officers, sustainability/energy teams, building owners, IT staff, and facility managers).

Every question was then independently graded by an LLM (GPT-5.5) on two separate difficulty axes: how hard the question is for a human to reason about, and how hard it is for an automated system to answer properly. The gap between these two scores is the central finding.

Who this is for: colleagues, reviewers, and anyone else who wants to understand what the corpus analysis found and what the numbers mean -- without needing to run the code themselves.

Section 0 -- Key Concepts and Terminology

Read this section before the results. Every term used later in the report is defined here.

0.1 The Two Axes: Surface vs. Latent Complexity

Every question in the corpus is rated on a 1-6 scale on two completely independent axes.

Axis 1: Surface Level (Cognitive Complexity)

What it measures: How hard is this question for a human to think about?

This captures the type of human reasoning the question requires -- recall, comparison, inference, causal thinking, or open-ended strategic reasoning. It does not depend on whether a computer can answer it; it is purely about the mental effort a person would expend.

Level	Name	What it means	Example question
L1	Factual Recall	Recall a single fact; trivial	"What color is the car?"
L2	Comparative Analysis	Compare two or more things	"Is this better than that?"
L3	Inferential Reasoning	Draw a conclusion from available information	"Based on the data, what's the trend?"
L4	Causal Diagnosis	Identify cause-and-effect	"Why is the system giving an error?"
L5	Systemic Synthesis	Reason across multiple interacting systems	"How do these systems interact?"
L6	Meta / Strategic Reasoning	Open-ended, goal-level, or technical	"How can we improve the building's energy efficiency?"

Key property: Surface level is rated on the phrasing and reasoning type of the question, not its data requirements. A

simple yes/no question asked in one sentence is usually L1 or L2 regardless of how hard it is to answer technically.

Axis 2: Latent Level (Architectural Complexity)

What it measures: How hard is it for an automated smart-building system to answer this question as well as a knowledgeable human expert would?

This captures what the system would need to do -- look up a fact, run a statistical analysis, fuse data from multiple sources, orchestrate a multi-step plan, or control physical equipment. It depends entirely on system effort, not on the difficulty of the question phrasing.

Level	Name	What the system needs to	Example question and why
L1	Factual Recall	Single ontology or metadata	"What type of sensors are in Room 402?"
L2	Comparative Analysis	A few reads + light aggregation	"Which of the rooms has the highest CO2 right now?"
L3	Inferential Reasoning	Multi-hop retrieval or derivation	"Which zones are overbooked at 02:00?"
L4	Causal Diagnosis	Time-series retrieval + statistical analysis	"Why are all the elevators stuck?"
L5	Systemic Synthesis	Multi-system data fusion	"How much energy will the building need today?"
L6	Meta / Strategic Reasoning	Cross-domain orchestration	"If learning, how is the control for 12:00?"

Critical property: Latent level is independent of surface level. A question can be L1 surface (sounds trivially simple to a human) yet L6 latent (requires machinery that doesn't exist yet). This independence is the entire reason for having two axes -- and the source of the main finding.

0.2 The Iceberg Effect

Definition: A question exhibits the iceberg effect when its latent (architectural) complexity is significantly higher than its surface (cognitive) complexity -- i.e., it looks easy but is hard to answer properly.

The name comes from an iceberg: only a small fraction is visible above the water (what the question looks like), while the vast majority is hidden below the surface (what it costs the system to answer truthfully).

Measurement in this study:

- * Any iceberg: latent level > surface level (63.3% of all questions)
- * Moderate iceberg: latent level >= surface level + 2 (35.2%)
- * Extreme iceberg: surface <= 2 AND latent >= surface + 3 (770 questions, 12.6%)

Concrete example of an extreme iceberg:

"Can the lifts 'learn' that the cafeteria floor is the priority destination at exactly 12:05 PM?"

- * Surface level: L1 -- It is phrased as a yes/no capability check. A human answers it in one second.
- * Latent level: L6 -- A truthful system answer requires: occupancy detection (sensor not present), time-series learning algorithm, BMS lift controller write-back, a closed-loop control loop, and an approval workflow. None of these exist yet.

The iceberg effect means that question phrasing is a misleading signal for system design. You cannot estimate how hard it is to build a conversational building interface by reading the questions -- you must look at what the system needs to do to answer them correctly.

0.3 Answerability Categories

Each question is assigned one of three answerability labels based on whether the current OntoSage system (deployed on Building 1, Abacws, Cardiff University) can answer it today:

Category	Meaning	Count	%
full	The system can answer this completely today, without	1,300	22.0%

partial	The system can give a useful but incomplete answer	207 (3.4%), data
requires_extension	The system cannot answer truthfully without a new data	4,152 (69.4%)

requires_extension does not mean "impossible" or "out of scope". It means the right answer exists but requires adding something that is not yet present in the deployed system. Every requires_extension question has a [NEW] marker in its capability-gap field naming exactly what is missing.

0.4 Answer Basis

Each question is also tagged with how the answer is sourced:

Basis	Meaning	Count
building-data	Needs this building's specific sensor readings, topology, or	4,000
hybrid	Applies a general norm (e.g., a comfort standard) to this bu	1,045
general-knowledge	Answerable from general world knowledge with no building dat	772

general-knowledge questions automatically receive latent_level = 1 and answerability = full because the system can answer them with a single LLM call -- no pipelines or sensors needed.

0.5 The [NEW] Marker and Capability Gaps

Every question tagged requires_extension contains one or more [NEW] <item> tags in its classification record. These tags name the specific missing component -- for example:

- * [NEW] occupancy sensor feed
- * [NEW] energy metering integration
- * [NEW] BMS write-back actuation

Aggregating all [NEW] items across 6,117 rows produces the capability-gap taxonomy in Section 4 and file T5_new_capability_gaps.csv. This taxonomy is the engineering backlog derived directly from what users actually ask.

0.6 Elicitation Stages

Questions were collected in four distinct stages to ensure diversity:

Stage	Method	Count	What it surfaces
Stage 1	Open-ended: "What would you ask your building?"	1,634	Unprompted, intuitive questions
Stage 2	Sensor-prompted: shown a list of sensor types	1,573	Data-aware questions about specific
Stage 3	Scenario-based: given a situation (e.g., "it's a hot day")	1,449	Contextual, diagnostic questions
Stage 4	Goal-oriented: given a goal (e.g., "reduce energy use")	1,309	Strategic, high-level intent questions

Stages 3 and 4 produce harder questions (higher surface and latent levels on average) because the scenarios and goals naturally invite causal and systemic thinking.

0.7 Persona Groups

Participants are grouped into 8 personas for analysis:

Persona	Who they are	Typical question style
Students / Researchers	University students and academic researchers	Research-oriented, exploratory, high-latent
Occupants / Tenants	Day-to-day building users	Comfort-focused, conversational, deceptively
Guests / Visitors	One-time or infrequent visitors	Wayfinding, simple factual, occasionally stra
Health & Safety Officers	Compliance and safety roles	Regulatory, procedural, predictive (evacuatio
Sustainability / Energy teams	Energy and net-zero specialists	Energy-saturated, deepest analytical/predicti

Building Owners	Decision-makers with strategic overview	High-level, policy, general-knowledge-heavy
IT / Data Scientists	Technical staff managing systems	Diagnostic, system-level, explicit
Facility Managers	Staff responsible for building operations	Operational, data-heavy, explicit

1. Executive Summary

Every question was graded on two independent 1-6 axes:

- * Surface (cognitive) level -- how hard the question is for a human to reason about.
- * Latent (architectural) level -- how hard it is for an automated system to answer it as well as a knowledgeable human would.

Headline	Value
Mean surface level	2.40 (most questions read as simple recall or inference)
Mean latent level	3.37 (most questions require at least multi-hop analytics to answer)
Questions architecturally harder than they look (latent > surface)	68.3%
Extreme "iceberg" questions (surface <= 2, latent >= 5)	770 (10.6%)
Fully answerable by the current bldg1 deployment	22.6%
Need a data/capability extension (not new reasoning)	74.0%
Latent-L6 questions (closed-loop autonomy / cross-domain)	645 (10.5%)
Surface-L6 questions (meta/strategic/ethical)	129 (2.1%)

The single most important result: the difficulty of human-building conversation is dominated by the latent axis, not the cognitive one. People phrase architecturally enormous requests as trivial yes/no questions. The gap between how questions look and what they cost to answer is the central design problem -- and it is invisible to any analysis that only looks at question phrasing.

2. The Complexity Iceberg (key figure: F2_level_vs_latent_heatmap.png)

The latent distribution is shifted a full level to the right of the surface/cognitive one (F1_level_distributions.png). The table below shows how many questions fall at each level on each axis:

Level	Level Name	Surface (cognitive) count	Latent (architectural) count
L1	Factual Recall	2,378	1,049
L2	Comparative Analysis	790	706
L3	Inferential Reasoning	1,971	1,490
L4	Causal Diagnosis	208	1,309
L5	Systemic Synthesis	641	918
L6	Meta / Strategic Reasoning	129	645

How to read this table: Only 208 questions read as requiring causal diagnosis (surface L4), but 1,309 require time-series analytics to answer properly (latent L4). The question "is my office always this stuffy?" is surface L1 -- it sounds like a one-word yes/no. But the truthful answer needs historical CO2 time-series, a trend analysis, a comparison to comfort thresholds, and a causal inference about HVAC scheduling. That is latent L4.

Similarly: only 129 questions read as meta-strategic (surface L6), but 645 require cross-domain orchestration to answer (latent L6). The phrasing dramatically underestimates system complexity.

770 extreme icebergs (T2_iceberg_questions.csv) look like simple capability checks but demand latent L5-L6 machinery. Representative examples:

"If water flow is detected for 2 hours straight in a bathroom at night, can the system automatically shut off that valve?" -- surface L1, latent L6

"Can the lifts 'learn' that the cafeteria floor is the priority destination at exactly 12:05 PM?" -- surface L1, latent L6

"Can the system predict a Peak Demand event from the afternoon weather forecast and start pre-cooling?" --

surface L1, latent L6

"Could escalators switch to standby when corridors are empty?" -- surface L1, latent L6

A human answers these "yes/no" in one second. A truthful system answer requires an event-condition-action rule engine, sensor feeds that do not exist yet, predictive models, and an actuation loop.

3. The Hardest Questions, in Four Archetypes -- and How We Solve Each

The top-60 composite-difficulty ranking is in T1_top60_hardest_questions.csv; the hardest question per persona in T7_hardest_per_persona.csv. They cluster into four archetypes:

Archetype A -- Autonomy Mandates (the genuinely hardest class)

"Can you make the building more eco-friendly?" (Occupant -- surface L6 / latent L6)

"Could we develop a way for our building to automatically shift large HVAC loads to off-peak times when more renewable energy is available?" (Occupant -- L5/L6)

"can you self manage yourself" (Building Owner -- L5/L6)

"How do I make you more environmentally-friendly?" (Student -- L6/L6)

These are not queries at all -- they are delegated open-ended goals. The user hands the system an objective ("be greener", "optimise", "self-manage") and expects it to decompose the goal, plan, act, and report. No retrieval pipeline answers these.

Solution path: goal-decomposition planning on top of the existing LangGraph multi-step planner. The planner node already decomposes compound queries; the extension is (1) a goal -> measurable sub-objectives decomposition step (e.g. "eco-friendly" -> energy/peak/waste/comfort KPIs), (2) per-objective routing into existing pipelines (analytics, forecast, recommend), and (3) an honest capability report: here is what I can measure today, here is what acting on it would require. Crucially OntoSage already routes actuation to a control intent that declines safely -- the right behaviour until BMS write-back is wired with RBAC guardrails. Full autonomy is the last phase of the roadmap (S6), not the first.

Archetype B -- Closed-Loop Automation Icebergs (largest hard class: 645 latent-L6 rows, T4_latent6_orchestration.csv)

The "can the building automatically X when Y" pattern above. Today, OntoSage's capability KB (Qdrant) can honestly answer whether the building can do it. Making the building actually do it requires:

- * An event-condition-action (ECA) rule layer -- a generic rules service (trigger: sensor condition; action: notify or actuate) configured per building in YAML, not code. ~80% of the 645 questions instantiate the same ECA template with different sensors/actions.
- * BMS write-back (1,009 question-mentions) behind RBAC control:write permission and an approval workflow.
- * The missing trigger feeds themselves (occupancy, water, energy -- see S4).

This is the strongest engineering argument in the corpus: one generic rule engine + one actuation gateway covers hundreds of distinct user wishes.

Archetype C -- Causal/Diagnostic Questions Blocked by Missing Data

"Why has energy use risen despite lower occupancy?"-type questions: surface L4, latent L5.

The reasoning machinery (analytics sandbox, anomaly detection, forecasting, correlation) already exists in OntoSage. These questions fail only because bldg1 has no energy meters, occupancy sensors, or weather feed. They need zero new reasoning code -- only data onboarding (S4). This is why "74.0% requires extension" is much less alarming than it sounds.

Archetype D -- Meta/Ethical/Social Questions (129 rows, T3_level6_meta_questions.csv)

"Would you be looking for people having an argument? Looking for signs of abuse? How would you step in?" (Occupant)

"Who is the judge of the results? How do you feel about this?" (Occupant)

"How does the building plan upgrades to meet future sustainability standards?" (Student)

Users probe the system's agency, judgement, and ethics -- surveillance boundaries, accountability, self-improvement. These are answered not with data but with policy: a per-building governance document (what the system monitors, what it will never infer, who reviews decisions) served through the capability KB. This is a deployment artifact (a YAML/markdown file in input/<bldg>/), not code -- and it is a publishable finding that real users ask these unprompted.

Honest caveat: a number of top-ranked rows are elliptical fragments ("can you optimize", "maintenance?", "emissions mimi"). The rubric correctly scores them highly ambiguous -> L6. The system answer is the existing clarification intent; the analysis answer is that ~2% of real queries are too underspecified to classify meaningfully, which itself motivates the clarification pipeline.

4. What Exactly Is Missing -- the Capability-Gap Taxonomy (T5, F4_new_gaps.png)

Every [NEW] item across 6,117 rows, canonicalised into a stable taxonomy. These are the things users need that the current system cannot provide:

Gap	Questions needing it	Type of fix
Occupancy sensing (incl. PIR/motion)	1,325 (21.7%)	new sensor feed
BMS actuation / write-back (incl. setpoint)	909 (15.5%)	integration + safety guardrails
Energy metering	933 (15.3%)	new sensor feed
Live streams, floors 0-4	855 (14.0%)	connect already-modelled sensors
Calendar / room-booking API	517 (8.5%)	external API (config)
Weather feed	502 (8.2%)	external API (config) -- free, keyless
Lighting sensing/control	424 (6.9%)	new feed
Access / security systems	415 (6.8%)	integration
Policy / document KB	333 (5.4%)	drop documents in input/ -- no code
Maintenance / CMMS records	323 (5.3%)	integration
Extended IAQ (PM/VOC/NO?)	323 (5.3%)	new sensors
Ontology metadata enrichment (capabilities, quiet zones)	307 (5.0%)	edit TTL/YAML -- no code
Noise / acoustic	285 (4.7%)	new sensors
Water metering	231 (3.8%)	new feed
Carbon / ESG data	151 (2.5%)	new feed / integration

Two structural findings:

- * Concentration: the top-8 gaps alone fully unblock 48.8% of all requires_extension questions. A handful of feeds, not hundreds of features, is the real backlog.
- * The long tail is irreducible: beyond the named categories sit ~5,000 mentions of niche needs whose most frequent item appears only 4 times (fire-alarm feed, gas meter, fridge temperature, Wi-Fi controller API, vending-machine inventory?). No amount of feature-building covers this tail. The only scalable answer is the architecture OntoSage already follows -- generic, config-driven onboarding (TTL + YAML + adapters) so each niche need is a data drop, not a development task. The tail is the empirical justification for the input-folder design.

Note also rows tagged "Ontology metadata enrichment" and "Policy / document KB" (637 questions combined): these need no new sensors and no new code -- only richer input files. They are the cheapest 10% win available.

5. Who Asks the Hard Questions? (counterintuitive -- F5_persona_difficulty.png, T6)

Persona	n	mean surface level	mean latent level	% requires extension	% general-knowledge
Sustainability / Energy	875	3.08	4.20	88.3	4.0
Students / Researchers	1,065	2.41	3.61	81.9	5.0
Occupants / Tenants	1,194	2.14	3.48	79.2	4.4
Health & Safety Officers	629	2.91	3.47	71.4	14.6
Guests / Visitors	1,378	2.14	3.28	75.2	9.6
Facility Managers	410	2.69	3.03	63.7	23.7
Building Owners	421	2.42	2.94	60.6	29.7
IT / Data Scientists	675	2.54	2.87	54.4	34.8

How to read this table: "mean surface level" is how hard the phrasing of their questions is; "mean latent level" is how hard it would be to answer them properly. "% requires extension" is the share that cannot be answered by the current system at all. "% general-knowledge" is the share that need no building data -- the LLM answers from general world knowledge.

The counterintuitive result holds, with a twist. Two groups demand the most architecture. The first is exactly the people with the least technical knowledge -- students, occupants and guests phrase high-latent automation wishes ("can the building just do X for me"). The second is the Sustainability / Energy specialists, who now top the latent scale (4.20) and the requires-extension share (88.3%): their net-zero, forecasting and optimisation questions land squarely on the data feeds bldg1 does not yet have (energy metering, occupancy, weather, carbon). By contrast the IT, Building-Owner and Facility-Manager professionals ask questions they could partly answer themselves -- conceptual, general-knowledge, lower system effort. Either way the system, not the user, must absorb the complexity -- direct evidence for the zero-knowledge-interaction design goal.

The naive assumption -- that general technical expertise predicts system stress -- does not hold: the heaviest demand comes from lay delegators and from domain specialists whose domain is the least instrumented, not from technically literate professionals in general. The design conclusion is the same: the system must absorb complexity because the people who stress it most are the least equipped (or least inclined) to do that work themselves.

6. Implementation Roadmap (mapped to the existing OntoSage architecture)

OntoSage today: LangGraph orchestrator (26 auto-registered intents), GraphDB Brick/BACnet/ASHRAE-223 ontology, MySQL time-series, sandboxed analytics, ARIMA/ETS forecasting, agentic RAG, Qdrant capability KB, floor-plan/spatial pipeline, report intake, multi-step planner, conversation memory. New intents are added via YAML + one node method -- no graph edits. Buildings are swapped by dropping files in input/<id>/.

The roadmap orders work by questions unblocked per unit of engineering, preserving the no-hardcoding principle (every phase ends in config, not building-specific code):

Phase	Work	Questions unblocked	Effort	New code?
1. Input enrichment	Richer TTL metadata	(room class), city, tenant days	easy, quiet zones)	none -- files only
2. Connect floors 0-4	Wire already-modelled BMS	Sensors to live MySQL	easy	none (UUIDs already modelled)
3. Generic external-floor computing	Event-driven adapter	to external systems	easy	once, generically
4. Sensor expansion	Occupancy, energy, IAQ, 500	requests to external BMS	easy	none beyond Phase 3
5. ECA rule engine	Generic event-condition-action	to external systems	easy	once, generically
6. BMS write-back + automation	Gateway to BMS	to external systems	easy	once, generically
7. Goal-decomposition	Multi-tier eco-friendly	to external systems	easy	once, generically

In parallel, the corpus directly supports the concept-layer ontology extending Brick (the human-vocabulary -> Brick-class -> analytic-recipe mapping, e.g. "stuffy" -> brick:CO2_Level_Sensor + threshold recipe). The 6,117 questions are the mining corpus for that vocabulary; because Brick classes are building-independent, the layer is authored once and inherited by every Brick-conformant building. That is the novel, publishable contribution; Phases

1-6 are the engineering substrate it runs on.

7. Anticipated Supervisor Questions -- Direct Answers

"What is the toughest question users asked?"

By compos
open-ended
and ethics,
the night-tim

"What made you think deeper?"

Three thing
phrasing is
architecture
need appea
scales.

"How will you solve it?"

Not per-que
tail; the fix i
every future
and (e) the
fully-answer

"How do you know the LLM classifications are trustworthy?"

Three layer
invariants v
sample with
(2) is done;

"Why two axes instead of one difficulty score?"

Because th
away the ic

"What does 'requires extension' actually mean -- is most of the corpus unanswerable?"

No. It mean
need occup
(analytics, f

8. Artifact Index (this folder)

File	Content
MASTER_ANALYSIS_REPORT.md	This report
00_data_quality_report.md	Invariant validation (0 violations)
01_summary_statistics.json	Every number in this report, machine-readable
F1_level_distributions.png	Surface vs latent histograms -- shows the rightward shift
F2_level_vs_latent_heatmap.png	The iceberg heatmap (lead figure) -- each cell = how many qu
F3_answerability_stack.png	Answerability share (full / partial / requires_extension) pe
F4_new_gaps.png	Capability-gap bar chart
F5_persona_difficulty.png	Difficulty by persona
F6_answer_basis.png	General-knowledge / building-data / hybrid split
T1_top60_hardest_questions.csv	Composite-difficulty ranking

T2_iceberg_questions.csv	All 770 extreme icebergs
T3_level6_meta_questions.csv	All 129 meta/strategic questions
T4_latent6_orchestration.csv	All 645 closed-loop/orchestration questions
T5_new_capability_gaps.csv	Canonicalised gap taxonomy
T6_difficulty_by_persona.csv	Per-persona difficulty profile
T7_hardest_per_persona.csv	Hardest single question per persona
complexity_master_table.csv	Full 6,117-row dataset with both axis scores

Regenerate everything: python "Survey analysis and results/scripts/K_master_table_analysis.py"